

Kubernetes HA Home Lab — Full Technical Runbook

A complete, self-contained runbook for building a production-grade Kubernetes HA cluster on a single VMware ESXi host. Every command, every config file, every expected output is included. A person with no prior context should be able to reproduce this lab end-to-end.

Table of Contents

- [Prerequisites](#1-prerequisites)
- [Architecture Overview](#2-architecture-overview)
- [Node Inventory](#3-node-inventory)
- [ESXi Virtual Switch Setup](#4-esxi-virtual-switch-setup)
- [pfSense Installation and Configuration](#5-pfsense-installation-and-configuration)
- [Clone Config Repo on Manager VM](#6-clone-config-repo-on-manager-vm)
- [BIND9 DNS Server](#7-bind9-dns-server)
- [NFS Server](#8-nfs-server)
- [HAProxy Load Balancer](#9-haproxy-load-balancer)
- [Manager VM and Ansible Setup](#10-manager-vm-and-ansible-setup)
- [Kubernetes Stack](#11-kubernetes-stack)
 - [11.1 Prepare All Nodes via Ansible](#111-prepare-all-nodes-via-ansible)
 - [11.2 Initialize Cluster on master1](#112-initialize-cluster-on-master1)
 - [11.3 Join master2 and master3](#113-join-master2-and-master3)
 - [11.4 Join Worker Nodes](#114-join-worker-nodes)
 - [11.5 Install Calico CNI](#115-install-calico-cni)
 - [11.6 Install MetalLB](#116-install-metallb)
 - [11.7 Install Kubernetes Dashboard](#117-install-kubernetes-dashboard)
 - [11.8 Install Metrics Server](#118-install-metrics-server)
 - [11.9 Install NFS Dynamic Provisioner](#119-install-nfs-dynamic-provisioner)
 - [11.10 Current Cluster Status](#1110-current-cluster-status)
- [Troubleshooting](#12-troubleshooting)
- [Next Steps — Kafka for Corelight](#13-next-steps--kafka-for-corelight)

[Quick Reference](#14-quick-reference)

1. Prerequisites

Hardware Requirements

Component Minimum This Lab ----- CPU 8 cores 16+ cores recommended RAM 64 GB 64 GB Storage 500 GB SSD 1 TB NVMe NICs 2 (vmnic0 + vmnic1)

Software and Accounts Required

VMware ESXi license (free tier works for labs)

pfSense 2.8.1 ISO — download from [<https://www.pfsense.org/download/>]
(<https://www.pfsense.org/download/>)

RHEL 9 ISO or CentOS Stream 9 ISO (for infra, lb-k8s, manager VMs)

Ubuntu 22.04 LTS ISO (for all 6 K8s nodes)

SSH client (Terminal on Mac, PuTTY on Windows)

`kubectl` installed on your workstation

Helm v3 installed on your workstation or manager VM

Mac Workstation Static Route

To reach the 172.25.25.0/24 network from your Mac (via pfSense WAN at 192.168.1.20):

```
# On your Mac — add persistent route for the K8s lab network
sudo route add -net 172.25.25.0/24 192.168.1.20
```

****Note:**** This route is not persistent across reboots. Add it to a login item script or `/etc/rc.local` equivalent if you want it permanent. On macOS you can use a LaunchDaemon plist under `/Library/LaunchDaemons/`.

Verify connectivity:

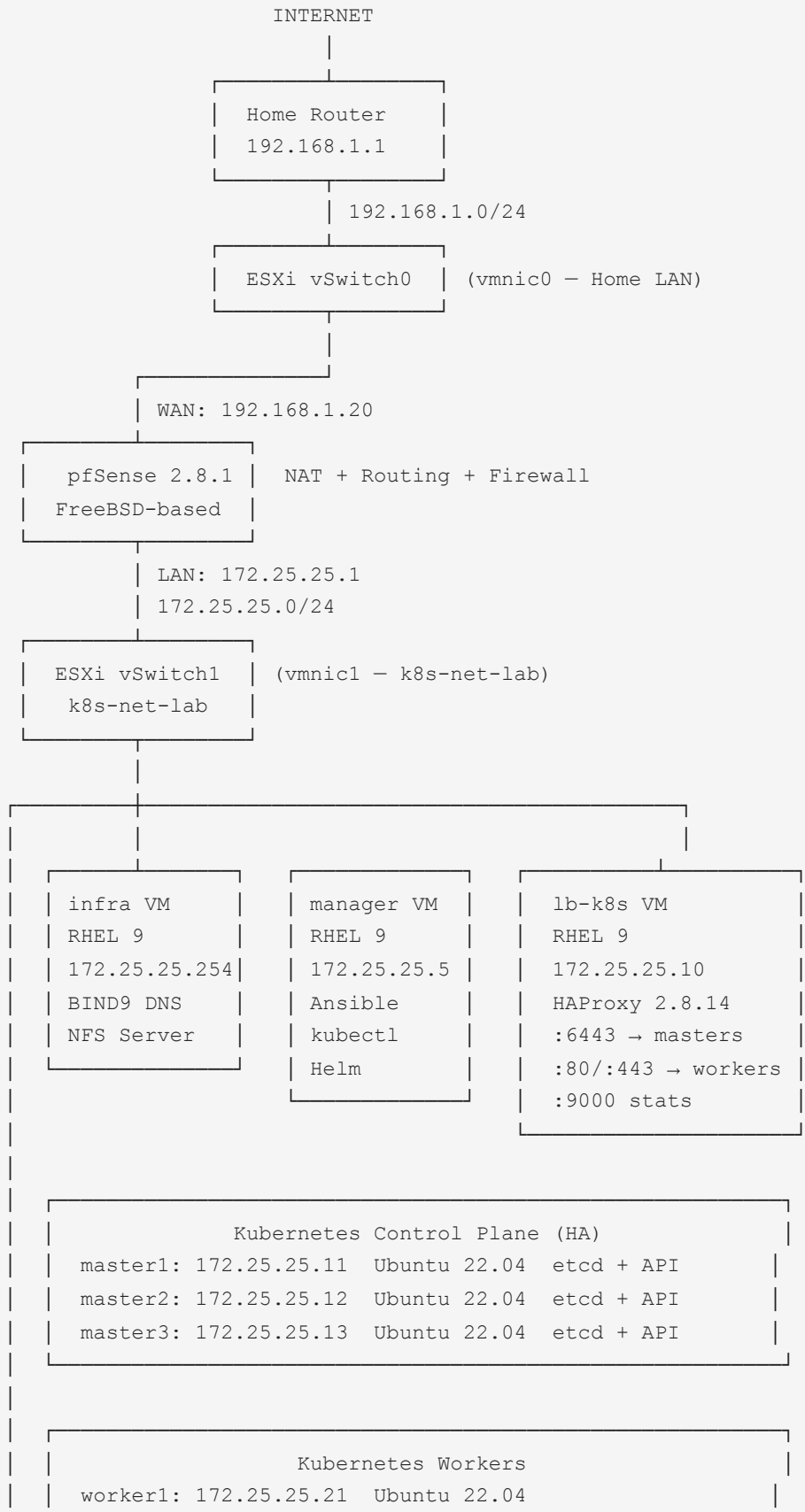
```
ping -c 3 172.25.25.1      # pfSense LAN
ping -c 3 172.25.25.254   # infra DNS/NFS
```

DNS on Your Mac

Add the lab DNS server so `.mo.lab.local` names resolve:

```
# Create a resolver file for the lab domain
sudo mkdir -p /etc/resolver
echo "nameserver 172.25.25.254" | sudo tee /etc/resolver/mo.lab.local
```

2. Architecture Overview



```
worker2: 172.25.25.22  Ubuntu 22.04
worker3: 172.25.25.23  Ubuntu 22.04
```

```
MetalLB IP Pool: 172.25.25.100 - 172.25.25.150
Pod CIDR:        10.244.0.0/16
Domain:          mo.lab.local
```

Traffic flow for external access:

Request hits `lb-k8s.mo.lab.local:6443` (HAProxy)

HAProxy round-robins across master1/2/3 port 6443 (Kubernetes API)

HTTP/HTTPS traffic on :80/:443 round-robins to worker nodes

MetalLB assigns IPs from 172.25.25.100-150 for LoadBalancer services

Dashboard accessible at `http://172.25.25.100` (MetalLB-assigned IP)

3. Node Inventory

Hostname	IP Address	OS Role	evCPU	RAM
pfsense	172.25.25.1 (LAN) / 192.168.1.20 (WAN)	Router/Firewall/NAT	24 GB	infra.mo.lab.local
FreeBSD (pfSense 2.8.1)	172.25.25.254	RHEL 9 BIND9 DNS + NFS	24 GB	manager.mo.lab.local
172.25.25.5	RHEL 9 Ansible + kubectl + Helm	24 GB	lb-k8s.mo.lab.local	
172.25.25.10	RHEL 9 HAProxy 2.8.14	24 GB	master1-k8s.mo.lab.local	
172.25.25.11	Ubuntu 22.04 LTS K8s Control Plane + etcd	48 GB	master2-k8s.mo.lab.local	
172.25.25.12	Ubuntu 22.04 LTS K8s Control Plane + etcd	48 GB	master3-k8s.mo.lab.local	
172.25.25.13	Ubuntu 22.04 LTS K8s Control Plane + etcd	48 GB	worker1-k8s.mo.lab.local	
172.25.25.21	Ubuntu 22.04 LTS K8s Worker	48 GB	worker2-k8s.mo.lab.local	
172.25.25.22	Ubuntu 22.04 LTS K8s Worker	48 GB	worker3-k8s.mo.lab.local	
172.25.25.23	Ubuntu 22.04 LTS K8s Worker	48 GB		

4. ESXi Virtual Switch Setup

Step 1 — Log into ESXi Host Client

Navigate to `https://` in a browser. Log in with root credentials.

Step 2 — Verify Physical NICs

Go to **Networking** → **Physical NICs**. Confirm:

`vmnic0` — connected to your home LAN switch (192.168.1.0/24)

`vmnic1` — connected to a dedicated switch or direct cable for the K8s lab network

Step 3 — Create vSwitch0 (Home LAN — already exists by default)

vSwitch0 is created automatically during ESXi installation and is bound to vmnic0. Confirm it exists under **Networking** → **Virtual Switches**.

Default port group: `VM Network` — VMs on this switch get 192.168.1.x IPs.

Step 4 — Create vSwitch1 (k8s-net-lab)

Go to **Networking** → **Virtual Switches** → **Add standard virtual switch**

Name: ``k8s-net-lab``

Uplink: ``vmnic1``

MTU: ``1500`` (default)

Click **Add**

Step 5 — Create Port Group on vSwitch1

Go to **Networking** → **Port groups** → **Add port group**

Name: ``k8s-net-lab``

VLAN ID: ``0`` (none)

Virtual switch: ``k8s-net-lab``

Click **Add**

Note: All K8s VMs (infra, manager, lb-k8s, master1-3, worker1-3) must have their primary NIC connected to the ``k8s-net-lab`` port group. pfSense gets two NICs — one on ``VM Network`` (WAN) and one on ``k8s-net-lab`` (LAN).

Step 6 — VM NIC Assignments Summary

VMNIC 1 (Port Group)	NIC 2 (Port Group)	-----	pfSense	VM Network (WAN)	k8s-net-lab (LAN)
infra	k8s-net-lab	—	manager	k8s-net-lab	—
lb-k8s	k8s-net-lab	—	master1/2/3	k8s-net-lab	—
worker1/2/3	k8s-net-lab	—			

Verify

In ESXi Host Client, each K8s VM should show its NIC connected to `k8s-net-lab`. pfSense should show two NICs.

5. pfSense Installation and Configuration

Step 1 — Download pfSense ISO

Download the AMD64 DVD Image (ISO) from the official pfSense download page:

```
https://www.pfsense.org/download/
Version: 2.8.1
Architecture: AMD64
Installer: DVD Image (ISO) Installer
```

Upload the ISO to an ESXi datastore: **Storage** → **Datastores** → **Datastore browser** → **Upload**.

Step 2 — Create pfSense VM in ESXi

****New Virtual Machine**** → ****Create a new virtual machine****

Name: `pfSense`

Guest OS family: ****Other**** / Guest OS version: ****FreeBSD 14 or later (64-bit)****

Storage: select your datastore

Customize settings:

- CPU: 2 vCPU
- Memory: 4 GB
- Hard disk: 20 GB (thin provisioned)
- **Network Adapter 1:** `VM Network` (WAN — connects to home LAN)
- **Add Network Adapter** → **Network Adapter 2:** `k8s-net-lab` (LAN)
- CD/DVD Drive: point to the uploaded pfSense ISO; check **Connect at power on**

Click ****Finish**** then ****Power On****

Step 3 — Boot and Initial Setup

In the ESXi console:

Accept the copyright notice

****Install pfSense**** → accept defaults → select disk → ****Auto (ZFS)**** or ****UFS**** → confirm

After installation completes, ****reboot**** and remove ISO from virtual CD

Step 4 — Assign Interfaces

On first boot, pfSense prompts for interface assignment:

```
Valid interfaces:
  em0  xx:xx:xx:xx:xx:xx  (vmnic0 side — home LAN)
  em1  xx:xx:xx:xx:xx:xx  (vmnic1 side — k8s-net-lab)

Do VLANs need to be set up first? n

Enter the WAN interface name: em0
```

```

Enter the LAN interface name: em1

The interfaces will be assigned as follows:
  WAN -> em0
  LAN -> em1

Do you want to proceed? y

```

pfSense will configure:

WAN: DHCP from your home router → should get 192.168.1.20 (set a DHCP reservation on your router for pfSense's MAC)

LAN: 192.168.1.1 (default — we change this next)

Step 5 — Set LAN IP to 172.25.25.1

From the console menu, select **2) Set interface(s) IP address:**

```

Enter the number of the interface to configure: 2 (LAN)
Enter the new LAN IPv4 address: 172.25.25.1
Enter the new LAN IPv4 subnet bit count: 24
For a WAN, enter the new LAN IPv4 upstream gateway address: (leave blank)
Do you want to enable the DHCP server on LAN? n

```

Step 6 — Access pfSense Web UI

From your Mac (with the static route added in Section 1):

```

URL: https://172.25.25.1
Username: admin
Password: pfsense

```

Complete the Setup Wizard:

Hostname: `pfsense`

Domain: `mo.lab.local`

Primary DNS: `172.25.25.254` (our BIND9 server)

Secondary DNS: `8.8.8.8`

WAN: Static IP `192.168.1.20`, subnet `24`, gateway `192.168.1.1`

LAN: `172.25.25.1` / `24` (already set)

Set a strong admin password

Step 7 — Configure NAT (Outbound)

Go to **Firewall** → **NAT** → **Outbound**:

Switch to ****Manual Outbound NAT rule generation****

Add a rule:

- Interface: WAN
- Source: 172.25.25.0/24
- Destination: any
- Translation: **Interface address** (WAN IP)

Save and Apply Changes

This allows all K8s VMs to reach the internet through pfSense NAT.

Step 8 — Firewall Rules

Go to **Firewall** → **Rules** → **LAN**:

The default `allow all from LAN` rule is sufficient for a lab. For production, restrict accordingly.

Go to **Firewall** → **Rules** → **WAN**:

Add a rule to allow inbound SSH from your home LAN to manage VMs (optional):

Action: Pass, Interface: WAN, Protocol: TCP, Source: `192.168.1.0/24`, Destination: WAN address, Port: 22

Step 9 — Add Static Route on Mac

```
# On your Mac — run after each reboot, or add to a LaunchDaemon
sudo route add -net 172.25.25.0/24 192.168.1.20
```

Verify pfSense

```
# From your Mac
ping -c 3 172.25.25.1          # pfSense LAN — expect replies
curl -k https://172.25.25.1    # pfSense web UI — expect HTML redirect
```

From any K8s VM, verify internet access:

```
ping -c 3 8.8.8.8
curl -s https://example.com | head -5
```

6. Clone Config Repo on Manager VM

Step 1 — Install Git


```
# On manager VM (172.25.25.5)
sudo dnf install -y git
```

Step 2 — Clone the Repository

```
# On manager VM
git clone https://github.com/mshgayar/k8s-pfsense-esxi.git /root/ansible-k8s
cd /root/ansible-k8s
ls -la
```

Expected output:

```
total 48
drwxr-xr-x  5 root root 4096 Jan  1 00:00 .
drwxr-x--- 12 root root 4096 Jan  1 00:00 ..
drwxr-xr-x  8 root root 4096 Jan  1 00:00 .git
-rw-r--r--  1 root root 2048 Jan  1 00:00 README.md
-rw-r--r--  1 root root  512 Jan  1 00:00 hosts
drwxr-xr-x  2 root root 4096 Jan  1 00:00 playbooks
drwxr-xr-x  2 root root 4096 Jan  1 00:00 manifests
```

Ansible Hosts File

The file `/root/ansible-k8s/hosts` contains:

```
[masters]
master1-k8s ansible_host=172.25.25.11
master2-k8s ansible_host=172.25.25.12
master3-k8s ansible_host=172.25.25.13

[workers]
worker1-k8s ansible_host=172.25.25.21 ansible_user=worker1-k8s
worker2-k8s ansible_host=172.25.25.22 ansible_user=worker2-k8s
worker3-k8s ansible_host=172.25.25.23 ansible_user=worker3-k8s

[k8s:children]
masters
workers

[k8s:vars]
ansible_user=msalah
ansible_become=yes
ansible_become_method=sudo
```

7. BIND9 DNS Server

All commands in this section run on `infra.mo.lab.local` (172.25.25.254) unless noted.

Step 1 — Install BIND9

```
# On infra VM
sudo dnf install -y bind bind-utils
sudo systemctl enable --now named
```

Step 2 — Configure named.conf

Edit `/etc/named.conf`. The key settings block should look like:

```
sudo vi /etc/named.conf
```

```
options {
    listen-on port 53 { 127.0.0.1; 172.25.25.254; };
    listen-on-v6 port 53 { ::1; };
    directory      "/var/named";
    dump-file       "/var/named/data/cache_dump.db";
    statistics-file "/var/named/data/named_stats.txt";
    memstatistics-file "/var/named/data/named_mem_stats.txt";
    recursing-file  "/var/named/data/named.recursing";
    secroots-file   "/var/named/data/named.secroots";
    allow-query     { localhost; 172.25.25.0/24; };
    recursion yes;
    forwarders {
        8.8.8.8;
        8.8.4.4;
    };
    dnssec-validation no;
    managed-keys-directory "/var/named/dynamic";
    pid-file "/run/named/named.pid";
    session-keyfile "/run/named/session.key";
};

logging {
    channel default_debug {
        file "data/named.run";
        severity dynamic;
    };
};

zone "." IN {
    type hint;
    file "named.ca";
};

include "/etc/named.rfc1912.zones";
```

```
include "/etc/named.root.key";
include "/etc/named/named.conf.local";
```

Step 3 — Configure named.conf.local

Create `/etc/named/named.conf.local` :

```
sudo mkdir -p /etc/named
sudo vi /etc/named/named.conf.local
```

```
zone "mo.lab.local" {
    type master;
    file "/etc/named/zones/db.mo.lab.local";
};

zone "25.25.172.in-addr.arpa" {
    type master;
    file "/etc/named/zones/db.172.25.25.rev";
};
```

Step 4 — Create Zone Directory

```
sudo mkdir -p /etc/named/zones
```

Step 5 — Forward Zone File

Create `/etc/named/zones/db.mo.lab.local` :

```
sudo vi /etc/named/zones/db.mo.lab.local
```

```
$TTL 86400
@   IN  SOA  infra.mo.lab.local. admin.mo.lab.local. (
                2024010101  ; Serial
                3600        ; Refresh
                1800        ; Retry
                604800       ; Expire
                86400 )     ; Minimum TTL

@           IN  NS   infra.mo.lab.local.

infra      IN  A     172.25.25.254
manager    IN  A     172.25.25.5
lb-k8s     IN  A     172.25.25.10
master1-k8s IN  A     172.25.25.11
master2-k8s IN  A     172.25.25.12
master3-k8s IN  A     172.25.25.13
```

```
worker1-k8s      IN A    172.25.25.21
worker2-k8s      IN A    172.25.25.22
worker3-k8s      IN A    172.25.25.23
dashboard-k8s    IN A    172.25.25.100
```

Step 6 — Reverse Zone File

Create `/etc/named/zones/db.172.25.25.rev`:

```
sudo vi /etc/named/zones/db.172.25.25.rev
```

```
$TTL 86400
@   IN  SOA infra.mo.lab.local. infra.mo.lab.local. (
        2024010101    ; Serial
        3600          ; Refresh
        1800          ; Retry
        604800        ; Expire
        86400 )       ; Minimum TTL

@   IN  NS  infra.mo.lab.local.

254   IN  PTR infra.mo.lab.local.
10    IN  PTR lb-k8s.mo.lab.local.
5     IN  PTR manager.mo.lab.local.
11    IN  PTR master1-k8s.mo.lab.local.
12    IN  PTR master2-k8s.mo.lab.local.
13    IN  PTR master3-k8s.mo.lab.local.
21    IN  PTR worker1-k8s.mo.lab.local.
22    IN  PTR worker2-k8s.mo.lab.local.
23    IN  PTR worker3-k8s.mo.lab.local.
100   IN  PTR dashboard-k8s.mo.lab.local.
```

Step 7 — Set Permissions and Restart

```
sudo chown -R named:named /etc/named/zones
sudo chmod 640 /etc/named/zones/*.local /etc/named/zones/*.rev
sudo named-checkconf
sudo named-checkzone mo.lab.local /etc/named/zones/db.mo.lab.local
sudo named-checkzone 25.25.172.in-addr.arpa /etc/named/zones/db.172.25.25.rev
sudo systemctl restart named
sudo systemctl status named
```

Step 8 — Open Firewall Port

```
sudo firewall-cmd --permanent --add-service=dns
sudo firewall-cmd --reload
```

Step 9 — Configure All VMs to Use This DNS Server

On every K8s VM (masters, workers, manager, lb-k8s), edit `/etc/resolv.conf` or use `nmcli`:

```
# Using nmcli (preferred on RHEL/Ubuntu with NetworkManager)
sudo nmcli con mod "Wired connection 1" ipv4.dns "172.25.25.254"
sudo nmcli con up "Wired connection 1"
```

Or directly:

```
echo "nameserver 172.25.25.254" | sudo tee /etc/resolv.conf
echo "search mo.lab.local" | sudo tee -a /etc/resolv.conf
```

Verify DNS

```
# On any VM in the lab
dig @172.25.25.254 master1-k8s.mo.lab.local
dig @172.25.25.254 -x 172.25.25.11
dig @172.25.25.254 google.com      # Test forwarding
```

Expected output for the first command:

```
;; ANSWER SECTION:
master1-k8s.mo.lab.local. 86400 IN A 172.25.25.11

;; Query time: 1 msec
;; SERVER: 172.25.25.254#53(172.25.25.254)
```

8. NFS Server

All commands run on `infra.mo.lab.local` (172.25.25.254).

Step 1 — Install NFS Utilities

```
# On infra VM
sudo dnf install -y nfs-utils
sudo systemctl enable --now nfs-server rpcbind
```

Step 2 — Create Export Directory

```
sudo mkdir -p /home/nfs/k8s
sudo chmod 777 /home/nfs/k8s
sudo chown nobody:nobody /home/nfs/k8s
```

Step 3 — Configure /etc/exports

```
sudo vi /etc/exports
```

```
/home/nfs/k8s    172.25.25.0/24(rw,sync,no_subtree_check,no_root_squash)
```

Step 4 — Apply and Verify

```
sudo exportfs -rav
sudo exportfs -v
sudo showmount -e localhost
```

Expected output:

```
exporting 172.25.25.0/24:/home/nfs/k8s
```

```
Export list for localhost:
/home/nfs/k8s 172.25.25.0/24
```

Step 5 — Open Firewall Ports

```
sudo firewall-cmd --permanent --add-service=nfs
sudo firewall-cmd --permanent --add-service=rpc-bind
sudo firewall-cmd --permanent --add-service=mountd
sudo firewall-cmd --reload
```

Verify NFS from Another VM

```
# On manager VM
showmount -e 172.25.25.254
sudo mount -t nfs 172.25.25.254:/home/nfs/k8s /mnt
touch /mnt/testfile
ls -la /mnt/testfile
sudo umount /mnt
```

9. HAProxy Load Balancer

All commands run on `lb-k8s.mo.lab.local` (172.25.25.10).

Step 1 — Install HAProxy

```
# On lb-k8s VM
sudo dnf install -y haproxy
sudo systemctl enable haproxy
```

Step 2 — Configure HAProxy

Replace the contents of `/etc/haproxy/haproxy.cfg`:

```
sudo vi /etc/haproxy/haproxy.cfg
```

```
global
    log /dev/log local0
    log /dev/log local1 notice
    chroot /var/lib/haproxy
    stats socket /run/haproxy/admin.sock mode 660 level admin expose-fd listeners
    stats timeout 30s
    user haproxy
    group haproxy
    daemon

defaults
    log global
    mode tcp
    option tcplog
    option dontlognull
    timeout connect 5000
    timeout client 50000
    timeout server 50000

frontend kubernetes-api
    bind *:6443
    mode tcp
    default_backend kubernetes-masters

backend kubernetes-masters
    mode tcp
    balance roundrobin
    option tcp-check
    server master1 172.25.25.11:6443 check fall 3 rise 2
    server master2 172.25.25.12:6443 check fall 3 rise 2
    server master3 172.25.25.13:6443 check fall 3 rise 2

frontend http-ingress
    bind *:80
    mode tcp
    default_backend http-workers

backend http-workers
```

```

mode tcp
balance roundrobin
server worker1 172.25.25.21:80 check fall 3 rise 2
server worker2 172.25.25.22:80 check fall 3 rise 2
server worker3 172.25.25.23:80 check fall 3 rise 2

frontend https-ingress
bind *:443
mode tcp
default_backend https-workers

backend https-workers
mode tcp
balance roundrobin
server worker1 172.25.25.21:443 check fall 3 rise 2
server worker2 172.25.25.22:443 check fall 3 rise 2
server worker3 172.25.25.23:443 check fall 3 rise 2

listen stats
bind *:9000
mode http
stats enable
stats uri /
stats refresh 10s

```

Step 3 — Start HAProxy

```

sudo haproxy -c -f /etc/haproxy/haproxy.cfg    # validate config
sudo systemctl start haproxy
sudo systemctl status haproxy

```

Step 4 — Open Firewall Ports

```

sudo firewall-cmd --permanent --add-port=6443/tcp
sudo firewall-cmd --permanent --add-port=80/tcp
sudo firewall-cmd --permanent --add-port=443/tcp
sudo firewall-cmd --permanent --add-port=9000/tcp
sudo firewall-cmd --reload

```

Verify HAProxy Stats Page

From your browser or Mac terminal:

```
curl http://172.25.25.10:9000/
```


Navigate to `http://172.25.25.10:9000` in a browser. You should see the HAProxy stats dashboard showing:

`kubernetes-api` frontend: 0 sessions (masters not yet up — will show green after K8s install)

`http-ingress` and `https-ingress` frontends listed

All backends initially shown as DOWN (expected until K8s is running)

10. Manager VM and Ansible Setup

All commands run on `manager.mo.lab.local` (172.25.25.5).

Step 1 — Install Ansible and Tools

```
# On manager VM
sudo dnf install -y ansible python3-pip git
pip3 install --user kubernetes openshift
ansible --version
```

Expected output includes `ansible [core 2.14+] .`

Step 2 — Generate SSH Key

```
# On manager VM as root (or the msalah user)
ssh-keygen -t ed25519 -C "manager-k8s-ansible" -f ~/.ssh/id_ed25519 -N ""
cat ~/.ssh/id_ed25519.pub
```

Step 3 — Copy SSH Key to All 6 K8s Nodes

```
# On manager VM
for node in 172.25.25.11 172.25.25.12 172.25.25.13 172.25.25.21 172.25.25.22 172.25.25.23; do
    ssh-copy-id -i ~/.ssh/id_ed25519.pub msalah@$node
done
```

****Note:**** This assumes the user `msalah` already exists on all 6 Ubuntu nodes. Create it during OS installation or via the console before this step.

Step 4 — Configure Passwordless Sudo on All K8s Nodes

Run on each of the 6 K8s VMs (or via a quick Ansible ad-hoc after initial SSH auth):

```
# On each master and worker node
echo "msalah ALL=(ALL) NOPASSWD:ALL" | sudo tee /etc/sudoers.d/msalah
```

```
sudo chmod 440 /etc/sudoers.d/msalah
```

Or use Ansible after SSH keys are in place:

```
# On manager VM
ansible -i /root/ansible-k8s/hosts k8s -m shell \
  -a "echo 'msalah ALL=(ALL) NOPASSWD:ALL' | sudo tee /etc/sudoers.d/msalah && sudo chm
od 440 /etc/sudoers.d/msalah" \
  --ask-pass --ask-become-pass
```

Step 5 — Test Ansible Connectivity

```
# On manager VM
cd /root/ansible-k8s
ansible -i hosts k8s -m ping
```

Expected output:

```
master1-k8s | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
master2-k8s | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
master3-k8s | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
worker1-k8s | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
worker2-k8s | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
worker3-k8s | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
```

Step 6 — Install kubectl and Helm on Manager

```
# On manager VM — install kubectl
curl -LO "https://dl.k8s.io/release/v1.30.14/bin/linux/amd64/kubectl"
```

```

chmod +x kubect1
sudo mv kubect1 /usr/local/bin/

# Install Helm
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash

kubect1 version --client
helm version

```

11. Kubernetes Stack

11.1 Prepare All Nodes via Ansible

Create the playbook `/root/ansible-k8s/playbooks/prepare-nodes.yml` :

```

---
- name: Prepare all Kubernetes nodes
  hosts: k8s
  become: yes
  tasks:

    # — System fundamentals —————
    - name: Update apt cache and upgrade packages
      apt:
        update_cache: yes
        upgrade: dist
        cache_valid_time: 3600

    - name: Install required packages
      apt:
        name:
          - apt-transport-https
          - ca-certificates
          - curl
          - gnupg
          - lsb-release
          - software-properties-common
          - nfs-common
        state: present

    # — Disable swap (required by kubelet) —————
    - name: Disable swap immediately
      command: swapoff -a

    - name: Remove swap entry from /etc/fstab
      replace:
        path: /etc/fstab

```

```

    regexp: '^(#[^].*?\sswap\s+sw\s+.*)$'
    replace: '# \1'

# — Kernel modules —————
- name: Load required kernel modules
  modprobe:
    name: "{{ item }}"
    state: present
  loop:
    - overlay
    - br_netfilter

- name: Persist kernel modules
  copy:
    dest: /etc/modules-load.d/k8s.conf
    content: |
      overlay
      br_netfilter

# — Sysctl settings —————
- name: Set sysctl parameters for Kubernetes
  sysctl:
    name: "{{ item.key }}"
    value: "{{ item.value }}"
    state: present
    reload: yes
  loop:
    - { key: 'net.bridge.bridge-nf-call-iptables', value: '1' }
    - { key: 'net.bridge.bridge-nf-call-ip6tables', value: '1' }
    - { key: 'net.ipv4.ip_forward', value: '1' }

# — containerd —————
- name: Add Docker apt GPG key
  apt_key:
    url: https://download.docker.com/linux/ubuntu/gpg
    state: present

- name: Add Docker apt repository
  apt_repository:
    repo: "deb [arch=amd64] https://download.docker.com/linux/ubuntu {{ ansible_distribution_release }} stable"
    state: present

- name: Install containerd
  apt:
    name: containerd.io
    state: present
    update_cache: yes

- name: Create containerd config directory

```

```

file:
  path: /etc/containerd
  state: directory

- name: Generate default containerd config
  shell: containerd config default > /etc/containerd/config.toml
  args:
    creates: /etc/containerd/config.toml

- name: Enable SystemdCgroup in containerd config
  replace:
    path: /etc/containerd/config.toml
    regexp: 'SystemdCgroup = false'
    replace: 'SystemdCgroup = true'

- name: Enable and restart containerd
  systemd:
    name: containerd
    state: restarted
    enabled: yes
    daemon_reload: yes

# — Kubernetes packages —————
- name: Add Kubernetes apt GPG key
  apt_key:
    url: https://pkgs.k8s.io/core:/stable:/v1.30/deb/Release.key
    state: present

- name: Add Kubernetes apt repository
  apt_repository:
    repo: "deb https://pkgs.k8s.io/core:/stable:/v1.30/deb/ ."
    state: present

- name: Install kubeadm, kubelet, kubectl
  apt:
    name:
      - kubeadm=1.30.14-*
      - kubelet=1.30.14-*
      - kubectl=1.30.14-*
    state: present
    update_cache: yes

- name: Hold Kubernetes packages at current version
  dpkg_selections:
    name: "{{ item }}"
    selection: hold
  loop:
    - kubeadm
    - kubelet
    - kubectl

```

```

- name: Enable kubelet service
  systemd:
    name: kubelet
    enabled: yes
    state: started

# — Hostname and DNS —————
- name: Set DNS server in resolv.conf
  copy:
    dest: /etc/resolv.conf
    content: |
      nameserver 172.25.25.254
      search mo.lab.local

```

Run the playbook:

```

# On manager VM
cd /root/ansible-k8s
ansible-playbook -i hosts playbooks/prepare-nodes.yml -v

```

This typically takes 5-10 minutes for all 6 nodes in parallel.

Verify Node Preparation

```

# On manager VM — quick check via Ansible
ansible -i hosts k8s -m shell -a "kubeadm version && containerd --version"

```

11.2 Initialize Cluster on master1

```

# On master1 (172.25.25.11) — SSH in from manager
ssh msalah@172.25.25.11

```

```

# On master1
sudo kubeadm init \
  --control-plane-endpoint "172.25.25.10:6443" \
  --upload-certs \
  --pod-network-cidr 10.244.0.0/16 \
  --apiserver-advertise-address 172.25.25.11 \
  --kubernetes-version v1.30.14

```

****Note:**** `--control-plane-endpoint` points to the HAProxy load balancer, not master1 directly. This is what makes the cluster truly HA — all kubeconfig files will use the VIP (HAProxy) address.

Expected output (save the join commands!):

```
[init] Using Kubernetes version: v1.30.14
[preflight] Running pre-flight checks
...
Your Kubernetes control-plane has initialized successfully!
```

To start using your cluster, you need to run the following as a regular user:

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

You can now join any number of control-plane nodes by copying certificate authorities and service account keys to each node and then running the following command as root:

```
kubeadm join 172.25.25.10:6443 --token <TOKEN> \
--discovery-token-ca-cert-hash sha256:<HASH> \
--control-plane --certificate-key <CERT_KEY>
```

Then you can join any number of worker nodes by running the following on each as root:

```
kubeadm join 172.25.25.10:6443 --token <TOKEN> \
--discovery-token-ca-cert-hash sha256:<HASH>
```

Set Up kubeconfig on master1

```
# On master1
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Copy kubeconfig to Manager VM

```
# On manager VM
mkdir -p ~/.kube
scp msalah@172.25.25.11:~/.kube/config ~/.kube/config

# Verify from manager
kubectl get nodes
```

Expected (master1 NotReady — CNI not yet installed):

NAME	STATUS	ROLES	AGE	VERSION
master1-k8s	NotReady	control-plane	2m	v1.30.14

11.3 Join master2 and master3

****Note:**** The `--certificate-key` in the join command is only valid for 2 hours. If it expires, regenerate with: ``sudo kubeadm init phase upload-certs --upload-certs`` on master1.

Join master2

```
# On master2 (172.25.25.12)
sudo kubeadm join 172.25.25.10:6443 --token <TOKEN> \
  --discovery-token-ca-cert-hash sha256:<HASH> \
  --control-plane --certificate-key <CERT_KEY> \
  --apiserver-advertise-address 172.25.25.12
```

Join master3

```
# On master3 (172.25.25.13)
sudo kubeadm join 172.25.25.10:6443 --token <TOKEN> \
  --discovery-token-ca-cert-hash sha256:<HASH> \
  --control-plane --certificate-key <CERT_KEY> \
  --apiserver-advertise-address 172.25.25.13
```

Fix Stale etcd Member (if reusing a VM)

If a master node was previously part of a cluster and you get etcd member errors:

```
# On master1 - list etcd members
sudo ETCDCTL_API=3 etcdctl \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  member list

# Remove the stale member (replace <MEMBER_ID> with the ID from above)
sudo ETCDCTL_API=3 etcdctl \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  member remove <MEMBER_ID>

# Then on the problematic master - clean and rejoin
sudo kubeadm reset -f
sudo rm -rf /etc/cni/net.d /var/lib/etcd
sudo kubeadm join 172.25.25.10:6443 --token <TOKEN> \
  --discovery-token-ca-cert-hash sha256:<HASH> \
  --control-plane --certificate-key <CERT_KEY> \
  --apiserver-advertise-address 172.25.25.1X
```

Verify Control Plane


```
# On manager VM
kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
master1-k8s	NotReady	control-plane	10m	v1.30.14
master2-k8s	NotReady	control-plane	5m	v1.30.14
master3-k8s	NotReady	control-plane	3m	v1.30.14

(All NotReady until CNI is installed.)

11.4 Join Worker Nodes

Generate a new join token if the original expired:

```
# On master1
sudo kubeadm token create --print-join-command
```

Run on each worker node:

```
# On worker1 (172.25.25.21)
sudo kubeadm join 172.25.25.10:6443 --token <TOKEN> \
  --discovery-token-ca-cert-hash sha256:<HASH>
```

```
# On worker2 (172.25.25.22)
sudo kubeadm join 172.25.25.10:6443 --token <TOKEN> \
  --discovery-token-ca-cert-hash sha256:<HASH>
```

```
# On worker3 (172.25.25.23)
sudo kubeadm join 172.25.25.10:6443 --token <TOKEN> \
  --discovery-token-ca-cert-hash sha256:<HASH>
```

Verify All Nodes Joined

```
# On manager VM
kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
master1-k8s	NotReady	control-plane	15m	v1.30.14
master2-k8s	NotReady	control-plane	10m	v1.30.14
master3-k8s	NotReady	control-plane	8m	v1.30.14
worker1-k8s	NotReady	<none>	2m	v1.30.14
worker2-k8s	NotReady	<none>	1m	v1.30.14
worker3-k8s	NotReady	<none>	1m	v1.30.14

All nodes are NotReady — this is correct and expected. CNI must be installed next.

11.5 Install Calico CNI

Why Calico over Flannel?

Calico provides:

BGP-based routing (no encapsulation overhead in routed networks)

Network policy enforcement (L3/L4 pod-level firewall rules)

Better observability and troubleshooting tools (`calicoctl`)

Production-grade HA with Felix and BIRD daemons per node

Flannel is simpler but lacks network policy support and uses VXLAN encapsulation by default.

```
# On manager VM
kubectl apply -f https://raw.githubusercontent.com/projectcalico/calico/v3.27.0/manifests/calico.yaml
```

Watch nodes become Ready:

```
kubectl get nodes -w
```

Expected output (within 2-3 minutes):

NAME	STATUS	ROLES	AGE	VERSION
master1-k8s	Ready	control-plane	20m	v1.30.14
master2-k8s	Ready	control-plane	15m	v1.30.14
master3-k8s	Ready	control-plane	13m	v1.30.14
worker1-k8s	Ready	<none>	7m	v1.30.14
worker2-k8s	Ready	<none>	6m	v1.30.14
worker3-k8s	Ready	<none>	6m	v1.30.14

Verify Calico pods:

```
kubectl get pods -n kube-system -l k8s-app=calico-node
```

NAME	READY	STATUS	RESTARTS	AGE
calico-node-2xk9p	1/1	Running	0	3m
calico-node-7bqmr	1/1	Running	0	3m
calico-node-9ptlv	1/1	Running	0	3m
calico-node-dkrts	1/1	Running	0	3m
calico-node-mj8nf	1/1	Running	0	3m
calico-node-q4xln	1/1	Running	0	3m

11.6 Install MetalLB

MetalLB provides LoadBalancer service type in bare-metal environments (no cloud provider). It uses L2 (ARP) mode here, advertising IPs from the pool via ARP on the K8s network.

```
# On manager VM
kubectl apply -f https://raw.githubusercontent.com/metallb/metallb/v0.14.5/config/manifests/metallb-native.yaml

# Wait for MetalLB pods to be ready
kubectl wait --namespace metallb-system \
  --for=condition=ready pod \
  --selector=app=metallb \
  --timeout=120s
```

Create the IP address pool configuration. Save as `/root/ansible-k8s/manifests/metallb-config.yaml`:

```
apiVersion: metallb.io/v1beta1
kind: IPAddressPool
metadata:
  name: k8s-ip-pool
  namespace: metallb-system
spec:
  addresses:
  - 172.25.25.100-172.25.25.150
---
apiVersion: metallb.io/v1beta1
kind: L2Advertisement
metadata:
  name: k8s-l2-advert
  namespace: metallb-system
spec:
  ipAddressPools:
  - k8s-ip-pool
```

Apply it:

```
kubectl apply -f /root/ansible-k8s/manifests/metallb-config.yaml
```

Verify MetalLB

```
kubectl get pods -n metallb-system
kubectl get ipaddresspools -n metallb-system
kubectl get l2advertisements -n metallb-system
```

Expected:

NAME	READY	STATUS	RESTARTS	AGE
controller-7d9d97dcb-xnfbr	1/1	Running	0	3m
speaker-4klmn	1/1	Running	0	3m
speaker-7pqrs	1/1	Running	0	3m
speaker-9wxyz	1/1	Running	0	3m

NAME	AUTO ASSIGN	AVOID BUGGY IPS	ADDRESSES
k8s-ip-pool	true	false	["172.25.25.100-172.25.25.150"]

NAME	IPADDRESSPOOLS	IPADDRESSPOOL SELECTORS	INTERFACES
k8s-l2-advert	["k8s-ip-pool"]		

11.7 Install Kubernetes Dashboard

```
# On manager VM
kubectl apply -f https://raw.githubusercontent.com/kubernetes/dashboard/v2.7.0/aio/deploy/recommended.yaml
```

Change the Dashboard service from ClusterIP to LoadBalancer so MetalLB assigns it an external IP:

```
kubectl patch svc kubernetes-dashboard \
-n kubernetes-dashboard \
-p '{"spec": {"type": "LoadBalancer"}}'
```

Verify it gets an IP from the MetalLB pool:

```
kubectl get svc -n kubernetes-dashboard
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT (S)
kubernetes-dashboard	LoadBalancer	10.96.123.45	172.25.25.100	443:31234/TCP
dashboard-metrics-scraper	ClusterIP	10.96.234.56	<none>	8000/TCP

The dashboard DNS entry `dashboard-k8s.mo.lab.local` resolves to `172.25.25.100`.

Create Admin Service Account and Token

```
# Create admin user
kubectl create serviceaccount dashboard-admin -n kubernetes-dashboard

# Bind to cluster-admin role
kubectl create clusterrolebinding dashboard-admin \
--clusterrole=cluster-admin \
--serviceaccount=kubernetes-dashboard:dashboard-admin
```

```
# Generate a token (valid 24h)
kubectl create token dashboard-admin -n kubernetes-dashboard --duration=86400s
```

Copy the token output. Access the dashboard at:

```
https://172.25.25.100
```

or

```
https://dashboard-k8s.mo.lab.local
```

Select **Token** authentication and paste the token.

****Note:**** The dashboard uses a self-signed certificate. Accept the browser warning for lab use.

11.8 Install Metrics Server

```
# On manager VM
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
```

The Metrics Server requires valid kubelet TLS certificates. In a lab with self-signed certs, patch it to skip TLS verification:

```
kubectl patch deployment metrics-server \
  -n kube-system \
  --type='json' \
  -p='[{"op": "add", "path": "/spec/template/spec/containers/0/args/-", "value": "--kubelet-insecure-tls"}]'
```

Wait for it to become ready:

```
kubectl rollout status deployment/metrics-server -n kube-system
```

Verify kubectl top

```
kubectl top nodes
```

Expected output:

NAME	CPU (cores)	CPU%	MEMORY (bytes)	MEMORY%
master1-k8s	112m	2%	1812Mi	22%
master2-k8s	98m	2%	1754Mi	21%

master3-k8s	105m	2%	1789Mi	22%
worker1-k8s	45m	1%	834Mi	10%
worker2-k8s	52m	1%	891Mi	11%
worker3-k8s	48m	1%	862Mi	10%

```
kubectl top pods -A
```

11.9 Install NFS Dynamic Provisioner

The NFS subdir external provisioner watches for PVCs with `storageClassName: nfs-storage` and automatically creates subdirectories under the NFS export for each PVC.

```
# On manager VM - add Helm repo
helm repo add nfs-subdir-external-provisioner \
  https://kubernetes-sigs.github.io/nfs-subdir-external-provisioner/
helm repo update
```

Install the provisioner, pointing at the infra NFS server:

```
helm install nfs-provisioner \
  nfs-subdir-external-provisioner/nfs-subdir-external-provisioner \
  --namespace nfs-provisioner \
  --create-namespace \
  --set nfs.server=172.25.25.254 \
  --set nfs.path=/home/nfs/k8s \
  --set storageClass.name=nfs-storage \
  --set storageClass.defaultClass=true \
  --set storageClass.reclaimPolicy=Retain \
  --set storageClass.archiveOnDelete=false
```

Fix nfs-common on Worker Nodes

The provisioner pod runs on a worker node and needs `nfs-common` to mount NFS. The Ansible prepare playbook already installs it, but verify:

```
ansible -i /root/ansible-k8s/hosts workers -m shell -a "dpkg -l nfs-common | tail -1"
```

Expected: `ii nfs-common ... (installed)`

If missing:

```
ansible -i /root/ansible-k8s/hosts workers -m apt -a "name=nfs-common state=present" --
become
```

Verify NFS Provisioner

```
kubectl get pods -n nfs-provisioner
kubectl get storageclass
```

Expected:

NAME	READY	STATUS	RESTARTS	AGE
nfs-provisioner-nfs-subdir-...	1/1	Running	0	2m

NAME	PROVISIONER	RECLAIMPOLICY	VOL
UMEBINDINGMODE	ALLOWVOLUMEEXPANSION	AGE	
nfs-storage	cluster.local/nfs-provisioner-nfs-subdir-...	Retain	Imme
diate	true	2m	

Test PVC and Pod

Create `/root/ansible-k8s/manifests/test-pvc.yaml` :

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: test-pvc
spec:
  accessModes:
    - ReadWriteMany
  storageClassName: nfs-storage
  resources:
    requests:
      storage: 1Gi
```

Create `/root/ansible-k8s/manifests/test-pod-dynamic-storage.yaml` :

```
apiVersion: v1
kind: Pod
metadata:
  name: test-pod
spec:
  containers:
    - name: test-container
      image: nginx
      volumeMounts:
        - name: test-volume
          mountPath: /usr/share/nginx/html
  volumes:
    - name: test-volume
      persistentVolumeClaim:
        claimName: test-pvc
```

Apply and verify:

```
kubectl apply -f /root/ansible-k8s/manifests/test-pvc.yaml
kubectl apply -f /root/ansible-k8s/manifests/test-pod-dynamic-storage.yaml

kubectl get pvc test-pvc
kubectl get pod test-pod
```

Expected:

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES
STORAGECLASS	AGE			
test-pvc	Bound	pvc-a1b2c3d4-...	1Gi	RWX
nfs-storage	30s			

NAME	READY	STATUS	RESTARTS	AGE
test-pod	1/1	Running	0	20s

Verify the directory was created on the NFS server:

```
# On infra VM
ls -la /home/nfs/k8s/
```

```
drwxrwxrwx  2 nobody nobody 4096 Jan  1 00:00 default-test-pvc-pvc-a1b2c3d4-...
```

Clean up:

```
kubectl delete pod test-pod
kubectl delete pvc test-pvc
```

11.10 Current Cluster Status

```
# On manager VM - full cluster status
kubectl get nodes -o wide
```

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP	EXTERNAL-IP	OS
-IMAGE		KERNEL-VERSION					
master1-k8s	Ready	control-plane	2d	v1.30.14	172.25.25.11	<none>	Ub
untu 22.04.4 LTS		5.15.0-112-generic					
master2-k8s	Ready	control-plane	2d	v1.30.14	172.25.25.12	<none>	Ub
untu 22.04.4 LTS		5.15.0-112-generic					
master3-k8s	Ready	control-plane	2d	v1.30.14	172.25.25.13	<none>	Ub
untu 22.04.4 LTS		5.15.0-112-generic					
worker1-k8s	Ready	<none>	2d	v1.30.14	172.25.25.21	<none>	Ub
untu 22.04.4 LTS		5.15.0-112-generic					
worker2-k8s	Ready	<none>	2d	v1.30.14	172.25.25.22	<none>	Ub
untu 22.04.4 LTS		5.15.0-112-generic					


```
worker3-k8s    Ready    <none>          2d    v1.30.14    172.25.25.23    <none>    Ub
untu 22.04.4 LTS    5.15.0-112-generic
```

```
kubect1 get pods -A
```

NAMESPACE	NAME	READY	STATUS	R
ESTARTS AGE				
kube-system	calico-kube-controllers-...	1/1	Running	0
2d				
kube-system	calico-node-2xk9p	1/1	Running	0
2d				
kube-system	calico-node-7bqmr	1/1	Running	0
2d				
kube-system	calico-node-9ptlv	1/1	Running	0
2d				
kube-system	calico-node-dkrts	1/1	Running	0
2d				
kube-system	calico-node-mj8nf	1/1	Running	0
2d				
kube-system	calico-node-q4xln	1/1	Running	0
2d				
kube-system	coredns-7db6d8ff4d-f8kbz	1/1	Running	0
2d				
kube-system	coredns-7db6d8ff4d-vzqcl	1/1	Running	0
2d				
kube-system	etcd-master1-k8s	1/1	Running	0
2d				
kube-system	etcd-master2-k8s	1/1	Running	0
2d				
kube-system	etcd-master3-k8s	1/1	Running	0
2d				
kube-system	kube-apiserver-master1-k8s	1/1	Running	0
2d				
kube-system	kube-apiserver-master2-k8s	1/1	Running	0
2d				
kube-system	kube-apiserver-master3-k8s	1/1	Running	0
2d				
kube-system	kube-controller-manager-master1-k8s	1/1	Running	2
2d				
kube-system	kube-controller-manager-master2-k8s	1/1	Running	0
2d				
kube-system	kube-controller-manager-master3-k8s	1/1	Running	0
2d				
kube-system	kube-proxy-...	1/1	Running	0
2d				
kube-system	kube-scheduler-master1-k8s	1/1	Running	2
2d				
kube-system	kube-scheduler-master2-k8s	1/1	Running	0
2d				

kube-system	kube-scheduler-master3-k8s	1/1	Running	0
2d				
kube-system	metrics-server-...	1/1	Running	0
1d				
metallb-system	controller-7d9d97dcb-xnfbr	1/1	Running	0
2d				
metallb-system	speaker-4klmn	1/1	Running	0
2d				
metallb-system	speaker-7pqrs	1/1	Running	0
2d				
metallb-system	speaker-9wxyz	1/1	Running	0
2d				
kubernetes-dashboard	dashboard-metrics-scraper-...	1/1	Running	0
1d				
kubernetes-dashboard	kubernetes-dashboard-...	1/1	Running	0
1d				
nfs-provisioner	nfs-provisioner-nfs-subdir-...	1/1	Running	0
1d				

kubectl get svc -A

NAMESPACE	NAME	TYPE	CLUSTER-IP	EXTERNAL-IP
default	kubernetes	ClusterIP	10.96.0.1	<none>
kube-system	kube-dns	ClusterIP	10.96.0.10	<none>
kube-system	metrics-server	ClusterIP	10.96.x.x	<none>
kubernetes-dashboard	dashboard-metrics-scraper	ClusterIP	10.96.x.x	<none>
kubernetes-dashboard	kubernetes-dashboard	LoadBalancer	10.96.123.45	172.25.25.100
metallb-system	webhook-service	ClusterIP	10.96.x.x	<none>

kubectl get namespaces

NAME	STATUS	AGE
default	Active	2d
kube-node-lease	Active	2d
kube-public	Active	2d
kube-system	Active	2d
kubernetes-dashboard	Active	1d
metallb-system	Active	2d
nfs-provisioner	Active	1d

12. Troubleshooting

Issue 1 — Nodes Stay NotReady After kubeadm init

Root cause: CNI not installed, or wrong pod CIDR configured.

Fix:

```
kubectl describe node master1-k8s | grep -A5 "Conditions:"
# Look for "NetworkReady=false"
# Ensure Calico was applied and calico-node pods are Running
kubectl get pods -n kube-system | grep calico
```

If calico-node pods are CrashLoopBackOff, check if the pod CIDR in Calico matches `10.244.0.0/16`:

```
kubectl get configmap calico-config -n kube-system -o yaml | grep cidr
```

Issue 2 — kubeadm join Fails: "certificate has expired"

Root cause: The `--certificate-key` from `kubeadm init` output is only valid for 2 hours.

Fix:

```
# On master1 - regenerate and upload certs
sudo kubeadm init phase upload-certs --upload-certs
# Use the new certificate key with the existing token
```

Issue 3 — etcd Member Already Exists Error

Root cause: The node was previously part of a cluster; etcd still has a stale member entry.

Fix:

```
# On a healthy master, list members
sudo ETCDCTL_API=3 etcdctl \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  member list

# Remove the stale entry
sudo ETCDCTL_API=3 etcdctl \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  member remove <STALE_MEMBER_ID>
```

```
# On the problem node - full reset
sudo kubeadm reset -f && sudo rm -rf /etc/cni/net.d /var/lib/etcd
```

Issue 4 — MetalLB Does Not Assign External IP (Pending)

Root cause: IPAddressPool or L2Advertisement not applied, or speaker pods not running on worker nodes.

Fix:

```
kubectl get pods -n metallb-system
kubectl describe svc kubernetes-dashboard -n kubernetes-dashboard
# Check Events section for MetalLB messages

# Reapply config
kubectl apply -f /root/ansible-k8s/manifests/metallb-config.yaml
```

Issue 5 — PVC Stuck in Pending

Root cause: NFS provisioner pod is not running, or `nfs-common` missing from worker nodes.

Fix:

```
kubectl describe pvc test-pvc      # Check Events
kubectl logs -n nfs-provisioner deployment/nfs-provisioner-nfs-subdir-...

# Install nfs-common on all workers if missing
ansible -i /root/ansible-k8s/hosts workers -m apt \
  -a "name=nfs-common state=present" --become
```

Issue 6 — HAProxy Shows All Backends DOWN Before K8s Install

Symptom: HAProxy stats page at `http://172.25.25.10:9000` shows red for all master backends.

Root cause: The Kubernetes API server is not yet running. This is expected before cluster init.

Fix: Not a fix — just proceed with cluster initialization. Backends will turn green once masters are up.

Issue 7 — containerd CRI Not Found / imagePullError

Root cause: `SystemdCgroup` not set to `true` in containerd config, causing cgroup driver mismatch.

Fix:

```
# On the affected node
sudo grep -n SystemdCgroup /etc/containerd/config.toml
# Must show: SystemdCgroup = true
# If false:
```

```
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd/config.toml
sudo systemctl restart containerd
```

Issue 8 — kubectl: connection refused (8080 or 6443)

Root cause: kubeconfig not set up for the current user, or pointing to wrong address.

Fix:

```
export KUBECONFIG=~/.kube/config
kubectl cluster-info
# If still failing, check the server address:
cat ~/.kube/config | grep server
# Should show: https://172.25.25.10:6443 (HAProxy)
```

Issue 9 — DNS Resolution Fails Inside Pods

Root cause: CoreDNS pods not running, or BIND9 forwarder not reachable from pod network.

Fix:

```
kubectl get pods -n kube-system -l k8s-app=kube-dns
# Should show 2x Running

# Test from a debug pod
kubectl run dns-debug --image=busybox:1.28 --rm -it -- nslookup kubernetes.default
kubectl run dns-debug --image=busybox:1.28 --rm -it -- nslookup google.com
```

If external DNS fails, check pfSense firewall rules allow DNS from pod CIDR.

Issue 10 — Metrics Server: "dial tcp: lookup master1-k8s"

Root cause: Metrics Server cannot resolve node hostnames. Nodes need to resolve each other's names.

Fix:

Ensure all nodes have `/etc/resolv.conf` pointing to `172.25.25.254` (BIND9). Also verify the `--kubelet-insecure-tls` patch was applied:

```
kubectl describe deployment metrics-server -n kube-system | grep insecure
```

Issue 11 — HAProxy Fails to Start: "cannot bind socket"

Root cause: Port 6443 or 80 already in use, or SELinux blocking the bind.

Fix:

```
# Check what's using the port
sudo ss -tlnp | grep 6443

# On RHEL, allow HAProxy to bind non-standard ports
sudo setsebool -P haproxy_connect_any 1
sudo systemctl restart haproxy
```

Issue 12 — pfSense NAT Not Working (K8s VMs Cannot Reach Internet)

Root cause: Outbound NAT rule missing or applied to wrong interface.

Fix:

In pfSense Web UI → **Firewall** → **NAT** → **Outbound**:

Confirm mode is ****Manual****

Confirm a rule exists: Source `172.25.25.0/24` → Interface `WAN` → Translation `Interface address`

Save and Apply

Also verify from a K8s VM:

```
tracert 8.8.8.8
# First hop should be 172.25.25.1 (pfSense LAN)
# Second hop should be 192.168.1.1 (home router)
```

13. Next Steps — Kafka for Corelight

The cluster is now operational and ready for production workloads. The planned next phase is deploying Apache Kafka to serve as a streaming pipeline for Corelight network sensor data.

Proposed Architecture

```
Corelight Sensor(s)
|
| JSON/Zeek logs over Kafka protocol
▼
Kafka Brokers (3x) — ZooKeeper or KRaft mode
Deployed on worker1/2/3 via StatefulSet
Persistent storage via NFS dynamic provisioner (nfs-storage)
|
| Topics: corelight.conn, corelight.dns, corelight.http, etc.
▼
Kafka Consumers:
- Elasticsearch/OpenSearch (indexing + search)
- Custom Python processors (enrichment, alerting)
- Grafana → visualization dashboards
```

Deployment Plan

****Install Strimzi Kafka Operator**** (Kubernetes-native Kafka management):

```
``bash
helm repo add strimzi https://strimzi.io/charts/
helm install strimzi-operator strimzi/strimzi-kafka-operator \
--namespace kafka --create-namespace
...
```

****Deploy Kafka cluster**** via Strimzi `Kafka` CRD:

- 3 broker replicas across worker1/2/3
- PersistentVolumes via `nfs-storage` StorageClass
- Expose via MetalLB LoadBalancer service for external Corelight producers

****Configure Corelight**** export target pointing to `172.25.25.10X` (MetalLB IP for Kafka external listener)

****Deploy monitoring****: Kafka Exporter → Prometheus → Grafana (all installable via Helm)

Storage Requirements

Each Kafka broker should have at minimum 50 GB of persistent storage. The NFS provisioner can satisfy this, though for high-throughput production use a dedicated block storage solution (Longhorn, Rook-Ceph) is recommended.

14. Quick Reference

All Service Endpoints

ServiceURL / AddressProtocol ----- pfSense Web UIhttps://172.25.25.1HTTPS pfSense WAN192.168.1.20— BIND9 DNS172.25.25.254:53DNS/UDP NFS Export172.25.25.254:/home/nfs/k8sNFS HAProxy Statshttp://172.25.25.10:9000HTTP Kubernetes APIhttps://172.25.25.10:6443HTTPS (via HAProxy) K8s Dashboardhttps://172.25.25.100HTTPS (MetalLB) K8s Dashboard DNShttps://dashboard-k8s.mo.lab.localHTTPS

All Component Versions

ComponentVersion ----- Kubernetesv1.30.14 kubeadm / kubelet / kubectlv1.30.14 Calico CNLv3.27.0 MetalLBv0.14.5 Kubernetes Dashboardv2.7.0 HAProxy2.8.14 pfSense2.8.1 containerdlatest stable Helmv3.x

Key Network Ranges

NetworkCIDR	Purpose	-----	Home LAN	192.168.1.0/24	External home network	K8s Lab
Network	172.25.25.0/24	All lab VMs	MetalLB Pool	172.25.25.100-150	LoadBalancer service IPs	Pod
CIDR	10.244.0.0/16	Kubernetes pod network (Calico)	Service CIDR	10.96.0.0/12	Kubernetes service	ClusterIPs

Essential kubectl Commands

```
# Cluster health
kubectl get nodes -o wide
kubectl get pods -A
kubectl get svc -A
kubectl top nodes
kubectl top pods -A

# Component status
kubectl get cs                                # deprecated but still works
kubectl get pods -n kube-system

# Calico
kubectl get pods -n kube-system -l k8s-app=calico-node
kubectl get pods -n kube-system -l app=calico-kube-controllers

# MetalLB
kubectl get ipaddresspools -n metallb-system
kubectl get l2advertisements -n metallb-system
kubectl get pods -n metallb-system

# Dashboard
kubectl get pods -n kubernetes-dashboard
kubectl get svc -n kubernetes-dashboard
kubectl create token dashboard-admin -n kubernetes-dashboard --duration=86400s

# NFS Provisioner
kubectl get pods -n nfs-provisioner
kubectl get storageclass
kubectl get pv
kubectl get pvc -A

# etcd health check (on any master)
sudo ETCDCTL_API=3 etcdctl \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  endpoint health
```



```
# Drain / uncordon a node for maintenance
kubectl drain worker1-k8s --ignore-daemonsets --delete-emptydir-data
kubectl uncordon worker1-k8s

# Force delete a stuck pod
kubectl delete pod <pod-name> --grace-period=0 --force

# Describe a failing resource
kubectl describe pod <pod-name> -n <namespace>
kubectl describe node <node-name>
```

Ansible Quick Commands

```
# From manager VM - /root/ansible-k8s/

# Ping all nodes
ansible -i hosts k8s -m ping

# Run a shell command on all masters
ansible -i hosts masters -m shell -a "kubectl get nodes" --become

# Run a shell command on all workers
ansible -i hosts workers -m shell -a "systemctl status containerd" --become

# Run the prepare playbook
ansible-playbook -i hosts playbooks/prepare-nodes.yml -v

# Run with specific tags
ansible-playbook -i hosts playbooks/prepare-nodes.yml --tags "packages,sysctl" -v
```

Static Route on Mac

```
# Add (needed after each reboot)
sudo route add -net 172.25.25.0/24 192.168.1.20

# Verify
netstat -nr | grep 172.25.25

# Delete if needed
sudo route delete -net 172.25.25.0/24
```

DNS Quick Tests

```
# Forward lookup
dig @172.25.25.254 master1-k8s.mo.lab.local +short

# Reverse lookup
```

```
dig @172.25.25.254 -x 172.25.25.11 +short

# External forwarding (should return Google's IPs)
dig @172.25.25.254 google.com +short

# From inside a pod
kubectl run dnstest --image=busybox:1.28 --rm -it -- nslookup kubernetes.default.svc.cluster.local
```

HAProxy Stats via CLI

```
# Check backend status via HAProxy admin socket (on lb-k8s VM)
echo "show stat" | sudo socat stdio /run/haproxy/admin.sock | cut -d',' -f1,2,18,19
```

Runbook maintained by msalah — last validated against K8s v1.30.14 / Calico v3.27.0 / MetalLB v0.14.5 on pfSense 2.8.1 / VMware ESXi.